

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Database Management Systems

COURSE CODE: CSA0509

COURSE OUTCOME COVERED

CO3: *Analyze relational databases using functional dependencies, normalization (1NF to 5NF), and key constraints to identify redundancy and ensure data integrity. (BL2, BL3)*

Activity Tool : Case Study (Medium)

Assessment Tool Weightage: 30%

QUESTIONS			Total Marks: 50
Q.No	Question	Bloom's Level	Marks
1	Analyze the case: An online shopping platform stores Order(OrderID, CustID, CustName, ProdID, ProdName, Qty, Price). Identify FDs and normalize to 3NF.	BL2 – Understand	5
2	A hospital maintains Patient(PID, PName, DocID, DocName, Specialization, WardNo, WardName). Identify anomalies and normalize to BCNF.	BL3 – Apply	5
3	Case study: A university stores Enrollment(SID, SName, CourseID, CourseName, Faculty, Grade). Analyze FDs and apply 2NF decomposition.	BL3 – Apply	5
4	Given the airline booking case: Booking(FlightNo, PassID, PassName, Seat, DeptCity, ArrCity, Date). Normalize up to BCNF.	BL3 – Apply	5
5	Analyze an HR system schema for a multinational company. Identify all functional and transitive dependencies and normalize to 3NF.	BL2 – Understand	5
6	Case: Library(MemberID, MemberName, BookID, Title, Author, Genre, BorrowDate). Identify MVDs and decompose to 4NF.	BL3 – Apply	5

7	Analyze a telecom billing schema and identify all candidate keys, primary keys, and functional dependencies before normalization.	BL2 – Understand	5
8	A retail inventory system has Product(PID, PName, SupplierID, SupplierName, Category, Price, Warehouse). Apply complete normalization.	BL3 – Apply	5
9	Study the given poorly normalized schema for a School ERP and propose a fully normalized design up to BCNF with justification.	BL3 – Apply	5
10	Given a movie streaming platform schema, identify join dependencies and decompose to 5NF with lossless-join verification.	BL3 – Apply	5

Code of Conduct:

I **ADHAVAN CHANDRASEKAR(192521209)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: ADHAVAN CHANDRASEKAR(192521209)

1. Analyze the case: An online shopping platform stores Order(OrderID, CustID, CustName, ProdID, ProdName, Qty, Price). Identify FDs and normalize to 3NF.

➤ Functional Dependencies

The major functional dependencies are:

- $\text{OrderID} \rightarrow \text{CustID}, \text{ProdID}, \text{Qty}$
- $\text{CustID} \rightarrow \text{CustName}$
- $\text{ProdID} \rightarrow \text{ProdName}, \text{Price}$

Candidate Key = OrderID

➤ 1NF

The relation is already in 1NF because all attributes contain atomic values.

Order(OrderID, CustID, CustName, ProdID, ProdName, Qty, Price)

➤ 2NF

Since the candidate key is a single attribute OrderID, there cannot be a partial dependency. Therefore, the relation is in 2NF.

➤ 3NF Decomposition

There are transitive dependencies:

$\text{OrderID} \rightarrow \text{CustID} \rightarrow \text{CustName}$

and

$\text{OrderID} \rightarrow \text{ProdID} \rightarrow \text{ProdName}, \text{Price}$

Therefore, decompose the relation into:

Customer

Customer(CustID, CustName)

Product

Product(ProdID, ProdName, Price)

Order

Order(OrderID, CustID, ProdID, Qty)

➤ Final 3NF Schema

Customer(CustID, CustName)

Product(ProdID, ProdName, Price)

Order(OrderID, CustID, ProdID,

Qty) Primary Keys:

- Customer → CustID
- Product → ProdID
- Order → OrderID

Foreign Keys:

- Order.CustID → Customer.CustID
- Order.ProdID → Product.ProdID

SQL Queries

MySQL 8.0 Command Line Client

```
mysql> CREATE TABLE Customer (  
->     CustID INT PRIMARY KEY,  
->     CustName VARCHAR(50)  
-> );  
Query OK, 0 rows affected (0.07 sec)  
  
mysql>  
mysql> CREATE TABLE Product (  
->     ProdID INT PRIMARY KEY,  
->     ProdName VARCHAR(100),  
->     Price DECIMAL(10,2)  
-> );  
Query OK, 0 rows affected (0.03 sec)  
  
mysql>  
mysql> CREATE TABLE Orders (  
->     OrderID INT PRIMARY KEY,  
->     CustID INT,  
->     ProdID INT,  
->     Qty INT,  
->     FOREIGN KEY (CustID) REFERENCES Customer(CustID),  
->     FOREIGN KEY (ProdID) REFERENCES Product(ProdID)  
-> );  
Query OK, 0 rows affected (0.16 sec)
```

2. A hospital maintains Patient(PID, PName, DocID, DocName, Specialization, WardNo, WardName). Identify anomalies and normalize to BCNF.

The functional dependencies are:

- $PID \rightarrow PName, DocID, WardNo$
- $DocID \rightarrow DocName, Specialization$
- $WardNo \rightarrow WardName$

Candidate Key:

Candidate Key = PID

2. Identification of Anomalies

There are transitive dependencies:

$PID \rightarrow DocID \rightarrow DocName, Specialization$

$PID \rightarrow WardNo \rightarrow WardName$

Therefore, storing all attributes in one relation causes redundancy.

3. BCNF Decomposition

A relation is in BCNF if for every non-trivial functional dependency:

$X \rightarrow Y$

X must be a superkey.

In the original relation:

$DocID \rightarrow DocName, Specialization$

but DocID is not a superkey.

Similarly:

$WardNo \rightarrow WardName$

but WardNo is not a superkey.

Therefore, the relation violates BCNF.

4. Decomposed Relations

Patient

Patient(PID, PName, DocID, WardNo)

Doctor

Doctor(DocID, DocName, Specialization)

Ward

Ward(WardNo, WardName)

SQL QUERY

MySQL 8.0 Command Line Client

```
mysql> CREATE TABLE Doctor (  
->     DocID INT PRIMARY KEY,  
->     DocName VARCHAR(50),  
->     Specialization VARCHAR(100)  
-> );  
Query OK, 0 rows affected (0.08 sec)  
  
mysql>  
mysql> CREATE TABLE Ward (  
->     WardNo INT PRIMARY KEY,  
->     WardName VARCHAR(50)  
-> );  
Query OK, 0 rows affected (0.04 sec)  
  
mysql>  
mysql> CREATE TABLE Patient (  
->     PID INT PRIMARY KEY,  
->     PName VARCHAR(50),  
->     DocID INT,  
->     WardNo INT,  
->     FOREIGN KEY (DocID) REFERENCES Doctor(DocID),  
->     FOREIGN KEY (WardNo) REFERENCES Ward(WardNo)  
-> );  
Query OK, 0 rows affected (0.08 sec)
```

3. Case study: A university stores Enrollment(SID, SName, CourseID, CourseName, Faculty, Grade). Analyze FDs and apply 2NF decomposition.

➤ **Functional Dependencies**

The functional dependencies are:

- $SID \rightarrow SName$
- $CourseID \rightarrow CourseName, Faculty$
- $(SID, CourseID) \rightarrow Grade$

Candidate Key:

(SID, CourseID)

➤ **1NF**

All attributes are atomic, so the relation is in 1NF.

➤ **2NF Analysis**

The candidate key is composite:

(SID, CourseID)

However:

$SID \rightarrow SName$

$CourseID \rightarrow CourseName, Faculty$

These are partial dependencies because non-key attributes depend on only part of the composite key.

Therefore, the relation is not in 2NF.

➤ **Decomposition into 2NF**

Student

Student(SID, SName)

Course

Course(CourseID, CourseName, Faculty)

Enrollment

Enrollment(SID, CourseID, Grade)

SQL QUERY:

```
mysql> CREATE TABLE Student (  
->     SID INT PRIMARY KEY,  
->     SName VARCHAR(50)  
-> );  
Query OK, 0 rows affected (0.10 sec)  
  
mysql>  
mysql> CREATE TABLE Course (  
->     CourseID INT PRIMARY KEY,  
->     CourseName VARCHAR(100),  
->     Faculty VARCHAR(100)  
-> );  
Query OK, 0 rows affected (0.04 sec)  
  
mysql>  
mysql> CREATE TABLE Enrollment (  
->     SID INT,  
->     CourseID INT,  
->     Grade CHAR(2),  
->     PRIMARY KEY (SID, CourseID),  
->     FOREIGN KEY (SID) REFERENCES Student(SID),  
->     FOREIGN KEY (CourseID) REFERENCES Course(CourseID)  
-> );  
Query OK, 0 rows affected (0.08 sec)
```


4. Given the airline booking case: Booking(FlightNo, PassID, PassName, Seat, DeptCity, ArrCity, Date). Normalize up to BCNF.

Assuming a passenger can book a particular flight and seat:

- $\text{PassID} \rightarrow \text{PassName}$
- $\text{FlightNo} \rightarrow \text{DeptCity}, \text{ArrCity}$
- $(\text{FlightNo}, \text{PassID}) \rightarrow \text{Seat}, \text{Date}$

Candidate Key:

(FlightNo, PassID)

2. BCNF Analysis

Consider:

$\text{PassID} \rightarrow \text{PassName}$

PassID is not a superkey of the complete Booking relation.

Also:

$\text{FlightNo} \rightarrow \text{DeptCity}, \text{ArrCity}$

FlightNo is not a superkey of the complete Booking relation.

Therefore, the relation violates BCNF.

3. BCNF Decomposition

Passenger

Passenger(PassID, PassName)

Flight

Flight(FlightNo, DeptCity, ArrCity)

Booking

Booking(FlightNo, PassID, Seat, Date)

SQL QUERY:

MySQL 8.0 Command Line Client

```
mysql> CREATE TABLE Passenger (  
->     PassID INT PRIMARY KEY,  
->     PassName VARCHAR(100)  
-> );  
Query OK, 0 rows affected (0.10 sec)  
  
mysql>  
mysql> CREATE TABLE Flight (  
->     FlightNo VARCHAR(20) PRIMARY KEY,  
->     DeptCity VARCHAR(50),  
->     ArrCity VARCHAR(50)  
-> );  
Query OK, 0 rows affected (0.04 sec)  
  
mysql>  
mysql> CREATE TABLE Booking (  
->     FlightNo VARCHAR(20),  
->     PassID INT,  
->     Seat VARCHAR(10),  
->     Date DATE,  
->     PRIMARY KEY (FlightNo, PassID),  
->     FOREIGN KEY (FlightNo) REFERENCES Flight(FlightNo),  
->     FOREIGN KEY (PassID) REFERENCES Passenger(PassID)  
-> );  
Query OK, 0 rows affected (0.15 sec)
```

5. Analyze an HR system schema for a multinational company. Identify all functional and transitive dependencies and normalize to 3NF.

The functional dependencies are:

$\text{EmpID} \rightarrow \text{EmpName}, \text{DeptID}, \text{Salary}$

$\text{DeptID} \rightarrow \text{DeptName}, \text{ManagerID}$

$\text{ManagerID} \rightarrow \text{ManagerName}$

Candidate Key:

EmpID

1. Transitive Dependencies

There are transitive dependencies:

$\text{EmpID} \rightarrow \text{DeptID} \rightarrow \text{DeptName}$

$\text{EmpID} \rightarrow \text{DeptID} \rightarrow \text{ManagerID}$

$\text{EmpID} \rightarrow \text{DeptID} \rightarrow \text{ManagerID} \rightarrow \text{ManagerName}$

Therefore, storing all information together causes redundancy.

2. 3NF Decomposition

Employee

Employee(EmpID, EmpName, DeptID, Salary)

Department

Department(DeptID, DeptName, ManagerID)

Manager

Manager(ManagerID, ManagerName)

SQL QUERY:

```
mysql> CREATE TABLE Manager (  
->     ManagerID INT PRIMARY KEY,  
->     ManagerName VARCHAR(100)  
-> );  
Query OK, 0 rows affected (0.10 sec)  
  
mysql>  
mysql> CREATE TABLE Department (  
->     DeptID INT PRIMARY KEY,  
->     DeptName VARCHAR(100),  
->     ManagerID INT,  
->     FOREIGN KEY (ManagerID) REFERENCES Manager(ManagerID)  
-> );  
Query OK, 0 rows affected (0.09 sec)  
  
mysql>  
mysql> CREATE TABLE Employee (  
->     EmpID INT PRIMARY KEY,  
->     EmpName VARCHAR(100),  
->     DeptID INT,  
->     Salary DECIMAL(10,2),  
->     FOREIGN KEY (DeptID) REFERENCES Department(DeptID)  
-> );  
Query OK, 0 rows affected (0.15 sec)
```

6. Case: Library(MemberID, MemberName, BookID, Title, Author, Genre, BorrowDate). Identify MVDs and decompose to 4NF.

Assume a member can borrow multiple books and a book can have multiple authors.

The following independent multivalued dependencies may exist:

$\text{MemberID} \twoheadrightarrow \text{BookID}$

$\text{BookID} \twoheadrightarrow \text{Author}$

The book's title and genre depend on BookID:

$\text{BookID} \rightarrow \text{Title, Genre}$

1. Problem

If a member borrows multiple books and each book has multiple authors, combining both independent multivalued facts creates unnecessary combinations.

For example:

Member 1 $\rightarrow$ Book A

Member 1 $\rightarrow$ Book B

and

Book A $\rightarrow$ Author X

Book A $\rightarrow$ Author Y

can produce redundant combinations.

2. Decomposition into 4NF

Member

Member(MemberID, MemberName)

Book

Book(BookID, Title, Genre)

BookAuthor

BookAuthor(BookID, Author)

Borrow

Borrow(MemberID, BookID, BorrowDate)

SQL QUERY:

 MySQL 8.0 Command Line Client

```
mysql> CREATE TABLE Member (  
->     MemberID INT PRIMARY KEY,  
->     MemberName VARCHAR(100)  
-> );  
Query OK, 0 rows affected (0.05 sec)  
  
mysql>  
mysql> CREATE TABLE Book (  
->     BookID INT PRIMARY KEY,  
->     Title VARCHAR(200),  
->     Genre VARCHAR(100)  
-> );  
Query OK, 0 rows affected (0.04 sec)  
  
mysql>  
mysql> CREATE TABLE BookAuthor (  
->     BookID INT,  
->     Author VARCHAR(100),  
->     PRIMARY KEY (BookID, Author),  
->     FOREIGN KEY (BookID) REFERENCES Book(BookID)  
-> );  
Query OK, 0 rows affected (0.04 sec)
```

```
mysql> CREATE TABLE Borrow (  
->     MemberID INT,  
->     BookID INT,  
->     BorrowDate DATE,  
->     PRIMARY KEY (MemberID, BookID, BorrowDate),  
->     FOREIGN KEY (MemberID) REFERENCES Member(MemberID),  
->     FOREIGN KEY (BookID) REFERENCES Book(BookID)  
-> );  
Query OK, 0 rows affected (0.14 sec)
```

7. Analyze a telecom billing schema and identify all candidate keys, primary keys, and functional dependencies before normalization.

The functional dependencies are:

$\text{StudentID} \rightarrow \text{StudentName}$

$\text{CourseID} \rightarrow \text{CourseName}, \text{FacultyID}$

$\text{FacultyID} \rightarrow \text{FacultyName}$

$(\text{StudentID}, \text{CourseID}) \rightarrow \text{Grade}$

1. Candidate Key

To determine all attributes:

From:

$\text{StudentID} \rightarrow \text{StudentName}$

$\text{CourseID} \rightarrow \text{CourseName}, \text{FacultyID}$

$\text{FacultyID} \rightarrow \text{FacultyName}$

and:

$(\text{StudentID}, \text{CourseID}) \rightarrow \text{Grade}$

we obtain:

$(\text{StudentID}, \text{CourseID})^+$

which gives:

StudentID

StudentName

CourseID

CourseName

FacultyID

FacultyName

Grade

Therefore:

Candidate Key = $(\text{StudentID}, \text{CourseID})$

2. Primary Key

The composite candidate key can be selected as the primary key:

PRIMARY KEY(StudentID, CourseID)

3. Prime Attributes

The prime attributes are:

StudentID

CourseID

The remaining attributes are non-prime attributes.

SQL

```
mysql> CREATE TABLE StudentCourse (  
    ->     StudentID INT,  
    ->     StudentName VARCHAR(100),  
    ->     CourseID INT,  
    ->     CourseName VARCHAR(100),  
    ->     FacultyID INT,  
    ->     FacultyName VARCHAR(100),  
    ->     Grade CHAR(2),  
    ->     PRIMARY KEY (StudentID, CourseID)  
    -> );  
Query OK, 0 rows affected (0.10 sec)
```

8. A retail inventory system has Product(PID, PName, SupplierID, SupplierName, Category, Price, Warehouse). Apply complete normalisation.

$PID \rightarrow PName, SupplierID, Category, Price, Warehouse$

$SupplierID \rightarrow SupplierName$

Candidate Key:

PID

1. Normalization Analysis

The dependency:

$SupplierID \rightarrow SupplierName$

creates a transitive dependency:

$PID \rightarrow SupplierID \rightarrow SupplierName$

Therefore, the original relation is not in 3NF.

2. Decomposition

Supplier

Supplier(SupplierID, SupplierName)

Product

Product(PID, PName, SupplierID, Category, Price, Warehouse)

SQL QUERY:

```
mysql> CREATE TABLE Supplier (  
->     SupplierID INT PRIMARY KEY,  
->     SupplierName VARCHAR(100)  
-> );  
Query OK, 0 rows affected (0.10 sec)  
  
mysql>  
mysql> CREATE TABLE Product (  
->     PID INT PRIMARY KEY,  
->     PName VARCHAR(100),  
->     SupplierID INT,  
->     Category VARCHAR(100),  
->     Price DECIMAL(10,2),  
->     Warehouse VARCHAR(100),  
->     FOREIGN KEY (SupplierID) REFERENCES Supplier(SupplierID)  
-> );  
Query OK, 0 rows affected (0.08 sec)
```

9. Study the given poorly normalized schema for a School ERP and propose a fully normalized design up to BCNF with justification. Poorly Normalized Relation

The dependencies are:

$\text{StudentID} \rightarrow \text{StudentName}, \text{ClassID}$

$\text{ClassID} \rightarrow \text{ClassName}, \text{TeacherID}$

$\text{TeacherID} \rightarrow \text{TeacherName}$

$\text{SubjectID} \rightarrow \text{SubjectName}$

$(\text{StudentID}, \text{SubjectID}) \rightarrow \text{Marks}$

Candidate Key:

$(\text{StudentID}, \text{SubjectID})$

1. Problems in the Original Relation

There are partial dependencies:

$\text{StudentID} \rightarrow \text{StudentName}, \text{ClassID}$

$\text{SubjectID} \rightarrow \text{SubjectName}$

There are also transitive dependencies:

$\text{StudentID} \rightarrow \text{ClassID} \rightarrow \text{ClassName}$

$\text{StudentID} \rightarrow \text{ClassID} \rightarrow \text{TeacherID} \rightarrow \text{TeacherName}$

Therefore, the relation is not in 2NF or 3NF.

2. BCNF Decomposition

Student

Student(StudentID, StudentName, ClassID)

Class

Class(ClassID, ClassName, TeacherID)

Teacher

Teacher(TeacherID, TeacherName)

Subject

Subject(SubjectID, SubjectName)

Marks

Marks(StudentID, SubjectID, Marks)

SQL QUERY:

 MySQL 8.0 Command Line Client

```
mysql> CREATE TABLE Teacher (  
->     TeacherID INT PRIMARY KEY,  
->     TeacherName VARCHAR(100)  
-> );  
Query OK, 0 rows affected (0.10 sec)  
  
mysql>  
mysql> CREATE TABLE Class (  
->     ClassID INT PRIMARY KEY,  
->     ClassName VARCHAR(100),  
->     TeacherID INT,  
->     FOREIGN KEY (TeacherID) REFERENCES Teacher(TeacherID)  
-> );  
Query OK, 0 rows affected (0.08 sec)  
  
mysql>  
mysql> CREATE TABLE Student (  
->     StudentID INT PRIMARY KEY,  
->     StudentName VARCHAR(100),  
->     ClassID INT,  
->     FOREIGN KEY (ClassID) REFERENCES Class(ClassID)  
-> );  
Query OK, 0 rows affected (0.07 sec)
```

```
mysql> CREATE TABLE Subject (  
->     SubjectID INT PRIMARY KEY,  
->     SubjectName VARCHAR(100)  
-> );  
Query OK, 0 rows affected (0.03 sec)  
  
mysql>  
mysql> CREATE TABLE Marks (  
->     StudentID INT,  
->     SubjectID INT,  
->     Marks INT,  
->     PRIMARY KEY (StudentID, SubjectID),  
->     FOREIGN KEY (StudentID) REFERENCES Student(StudentID),  
->     FOREIGN KEY (SubjectID) REFERENCES Subject(SubjectID)  
-> );  
Query OK, 0 rows affected (0.07 sec)
```

10. Given a movie streaming platform schema, identify join dependencies and decompose to 5NF with lossless-join verification. Given Schema

1. Multivalued Dependencies

The following multivalued dependencies exist:

MovieID $\twoheadrightarrow$ ActorID

MovieID $\twoheadrightarrow$ GenreID

This means that actors and genres are independently associated with a movie.

2. Problem with the Relation

Suppose:

Movie M1

Actors: A1, A2

Genres: G1, G2

The combined relation may require:

M1 A1 G1

M1 A1 G2

M1 A2 G1

M1 A2 G2

This creates redundant combinations.

3. Decomposition

Separate the independent relationships:

Movie

Movie(MovieID, MovieName)

Actor

Actor(ActorID, ActorName)

Genre

Genre(GenreID, GenreName)

MovieActor

MovieActor(MovieID, ActorID)

MovieGenre

MovieGenre(MovieID, GenreID)

SQL QUERY:

 MySQL 8.0 Command Line Client

```
mysql> CREATE TABLE Movie (  
->     MovieID INT PRIMARY KEY,  
->     MovieName VARCHAR(200)  
-> );  
Query OK, 0 rows affected (0.10 sec)  
  
mysql>  
mysql> CREATE TABLE Actor (  
->     ActorID INT PRIMARY KEY,  
->     ActorName VARCHAR(100)  
-> );  
Query OK, 0 rows affected (0.04 sec)  
  
mysql>  
mysql> CREATE TABLE Genre (  
->     GenreID INT PRIMARY KEY,  
->     GenreName VARCHAR(100)  
-> );  
Query OK, 0 rows affected (0.03 sec)  
  
mysql>  
mysql> CREATE TABLE MovieActor (  
->     MovieID INT,  
->     ActorID INT,  
->     PRIMARY KEY (MovieID, ActorID),  
->     FOREIGN KEY (MovieID) REFERENCES Movie(MovieID),  
->     FOREIGN KEY (ActorID) REFERENCES Actor(ActorID)  
-> );  
Query OK, 0 rows affected (0.07 sec)
```

```
mysql> CREATE TABLE MovieGenre (  
->     MovieID INT,  
->     GenreID INT,  
->     PRIMARY KEY (MovieID, GenreID),  
->     FOREIGN KEY (MovieID) REFERENCES Movie(MovieID),  
->     FOREIGN KEY (GenreID) REFERENCES Genre(GenreID)  
-> );  
Query OK, 0 rows affected (0.14 sec)
```


